TECHNICAL DOCUMENTATION EXAMINER VIVA READY PRODUCTION FULL-STACK SAAS

CareerForge — AI Resume & Portfolio Studio

Comprehensive System Architecture, Workflow Pipeline & Technical Viva Examination Guide

Project Type	Frontend / Backend	Database & ORM	AI Integration
Full-Stack SaaS Web App	Next.js 16 (App Router) + TS	Prisma 6.4 + SQLite	Google Gemini LLM Engine

1. Project Abstract & Core Deliverables

CareerForge is an enterprise-grade AI-powered resume and portfolio engineering studio. The platform solves the prevalent challenges candidates encounter when applying to modern organizations:

- **Applicant Tracking System (ATS) Gatekeeping**: Over 75% of resumes are rejected by algorithmic parsers before reaching a human recruiter due to non-compliant layouts, missing keywords, and unquantified bullet points.
- **Design vs. Readability Paradox**: Candidates either build beautiful but unparseable graphic resumes, or drab standard text files that fail to impress human hiring managers.
- **Web Portfolio Disconnect**: Creating a personal developer portfolio typically requires configuring hosting, repositories, and custom CSS, leading to abandoned or outdated personal sites.

CareerForge bridges these gaps with a synchronized dual-column real-time editor, Google Gemini AI integration using the Google XYZ formula, heuristic resume parsing, 8 professionally designed ATS & Pinterest-trending templates, native vector PDF and Word exports, and 1-click personal portfolio generation with public SSR routes.

2. System Architecture & Tech Stack

The platform is architected around the Next.js 16 App Router using a modular server-client boundary.



```
|      PRISMA ORM       |             |   GOOGLE GEMINI 2.5 LLM   |             |   STRIPE PAYMENTS   |
|  - SQLite (dev.db)    |             |   - Multi-Turn Summaries  |             |   - INR (paise: 5900)|
|  - 7 Relational Models|             |   - Heuristic Fallback    |             |   - Webhook Signature   |
+-----------------------+             +---------------------------+             +---------------------+
```

Layer	Technologies	Justification & Architectural Role
Frontend	Next.js 16, React 19, Tailwind CSS v4, Lucide Icons	App Router enables granular React Server Components (RSC) for lightning-fast initial page loads while providing responsive client-side interactivity on the live split-pane resume canvas.
Backend API	Next.js Server Route Handlers	Zero-cold-start API endpoints running in Node.js runtime, enforcing strict type-safe schemas and protecting sensitive API keys (Gemini, Stripe) from browser exposure.
Database & ORM	Prisma 6.4 + SQLite	Provides declarative schema modeling, type-safe query generation, foreign key cascades, and automated schema synchronization without manual SQL scripting.
Authentication	Jose (JWT) + BcryptJS	Stateless authentication using secure HTTP-only cookies prevents Cross-Site Scripting (XSS) token theft. Bcrypt provides 12-round salted password hashing.
AI Engine	Google Gemini 2.5 Flash + Fallback Engine	Generates professional summaries, rewrites accomplishment bullets according to Google's XYZ formula, and extracts job-specific ATS keywords with graceful offline heuristic fallbacks.
Document Export	Native Print CSS + `docx` library	Guarantees true vector PDF output with selectable text and exact page layout, alongside native Microsoft Word OpenXML (`.docx`) file compilation.
Monetization	Stripe API (INR ₹59 / month)	Enforces plan entitlements (Free vs. Pro) on the server, creates Stripe Checkout sessions with Indian Rupee (paise) support, and verifies payment statuses.

3. End-to-End User & Data Workflow

Stage 1: Candidate Authentication & Profile Initialization

When a user registers or logs in at `/auth/login`, their password is verified using `bcrypt.compare()`. Upon successful validation, the server generates a signed JSON Web Token (JWT) with HS256 encryption via the `jose` library and attaches it as an `httpOnly`, `sameSite: "lax"` cookie named `careerforge_session`. This guarantees the client JavaScript cannot access the raw session token.

Stage 2: Document Creation & Heuristic Parsing

The candidate can either create a resume from scratch or drag-and-drop an existing PDF/DOCX file. The upload endpoint (`/api/resumes/upload`):

1. Validates file MIME type and size (≤ 5MB).
2. Parses document streams using `pdf-parse` (for PDF) or `mammoth` (for DOCX).
3. Executes heuristic regular expression matching to isolate candidate name, phone number, email address, LinkedIn profile, GitHub handle, education degrees, and work experiences.
4. Computes a completeness score and returns pre-populated structured JSON into the editor.

Stage 3: Real-Time Split-Pane Editing & Live Rendering

The editor at `/resumes/[id]` displays a split viewport:

- **Left Pane (Control Deck)**: Section tabs (Personal Info, Summary, Experience, Education, Skills, Projects, Certifications, Languages, Custom Sections). As the user inputs text, state is managed in React memory.
- **Right Pane (Live Canvas)**: Renders the active template in real time. Changes in font size, accent color (Indigo, Emerald, Rose, Slate, Violet, Amber), page format (A4 / US Letter), and typography style (Sans, Serif, Mono) re-render instantly without full page reloads.
- **Autosave Controller**: Debounces keystrokes by 1,200ms before dispatching an asynchronous `PUT` request to `/api/resumes/[id]`.

Stage 4: AI Writing Assistant & ATS Tailoring Engine

When the candidate clicks **"Improve with AI"** on an experience bullet or **"ATS Tailor"** with a target job description:

1. The server invokes Google Gemini API with system instructions enforcing the **Google XYZ Formula**: *"Accomplished [X], measured by [Y], by doing [Z]"*.
2. The ATS Scanner extracts unigrams and bigrams from both the resume and the job description, removes standard English stopwords, calculates exact and fuzzy keyword overlap, and generates a match score (0-100%).
3. If the external Gemini API is unreachable or rate-limited, CareerForge seamlessly activates its internal rule-based heuristic rewriting algorithm, ensuring zero user-facing downtime.

Stage 5: High-Fidelity Export Pipeline

- **PDF Export**: Invokes native browser vector printing (`window.print()`) with custom `@media print` CSS rules that isolate the resume canvas, hide UI headers and sidebars, force exact millimeter margins, and prevent split lines across page boundaries.
- **DOCX Export**: The server route `/api/resumes/[id]/export/docx` constructs a native Microsoft Word OpenXML document using the `docx` library, formatting bold titles, bullet lists, and section dividers natively.

Stage 6: 1-Click Portfolio Website Publishing

With one click on **"Publish Portfolio"**, the application transforms the candidate's structured resume data into a responsive personal website deployed at `/p/[slug]`. The portfolio features 4 interchangeable themes (Minimalist Slate, Tech Dark Obsidian, Editorial Serif, Creative Warmth), a working contact form, social links, and an automated view analytics tracker.

4. Deep Dive: Pinterest-Trending Templates Engine

CareerForge implements 8 distinctive resume templates, with exclusive focus on current Pinterest, Behance, and Dribbble design trends for modern professional resumes:

A Swiss Split Sidebar Template (`swiss-sidebar`) — #1 Viral Layout on Pinterest

Visual Architecture: An asymmetrical two-column layout consisting of a tinted 33% width left column and a 67% white/neutral right column.
Left Column Elements: Circular monogram avatar with candidate initials, stacked contact icons (Email, Phone, Location, LinkedIn, GitHub), education timeline, categorized skill badges, spoken languages with visual proficiency bars, and certifications.
Right Column Elements: Bold modern sans-serif masthead, italicized executive pull-quote summary, and an **interactive connected vertical timeline rail** where each job experience is represented by an accent-colored circle node connected by an unbroken vertical SVG line.

B Quiet Luxury Editorial Template (`editorial-luxury`) — Haute-Couture Roman Serif

Visual Architecture: Inspired by high-end architectural and fashion editorial publications (Vogue, Kinfolk, Architectural Digest).
Typographic System: Roman serif typography (Playfair Display / Cormorant Garamond), wide letter-spacing (`tracking-[0.25em]`), and centered header hierarchy.
Numbered Index System: Each section is identified by a prominent numeric index: `01 / EXECUTIVE SUMMARY` , `02 / CAREER TRAJECTORY` , `03 / ACADEMIC CREDENTIALS` , `04 / VENTURES & PROJECTS` .

C Nordic Clean Template (`nordic-clean`) — Bauhaus Asymmetric Minimalist

Visual Architecture: Functional Bauhaus design with stark contrast, sharp rectangular accent indicators, and structured multi-column grid layouts for quick skimming by senior engineering leadership and recruiters.

5. Step-by-Step Live Viva Demonstration Script

Follow this exact sequence when demonstrating CareerForge to the examiner:

> **Examiner Demo Checklist:**
>
> 1. **Login & Authentication**: Navigate to `http://localhost:3000/auth/login` and sign in with `demo@careerforge.io` / `CareerForge123!` . Highlight HTTP-only cookie security.
> 2. **Dashboard Navigation**: Show the active resumes list, portfolio links, ATS score indicators, and usage metrics.
> 3. **Resume Editor & Live Split-Pane**: Open a resume. Type into the "Full Name" field and show how the live canvas on the right updates instantly in real time without lag.
> 4. **Pinterest Templates**: In the template selector, switch to **Swiss Split Sidebar**. Point out the monogram badge, tinted sidebar, and connected timeline rail. Then switch to **Quiet Luxury Editorial** to show the numbered magazine headers.
> 5. **AI Assistant (Google XYZ Formula)**: Click "Improve with AI" next to an experience bullet. Show how the passive bullet is rewritten with quantified metrics and action verbs.

6. **Job Description Tailoring**: Open the ATS Tailor tool, paste a sample job description, and run the scan. Show the 0-100% match score and the extracted missing keywords.
7. **Document Export**: Click "Export PDF" to show clean vector PDF generation with zero broken margins. Then click "Word DOCX" to demonstrate native file generation.
8. **Live Portfolio Website**: Click "View Portfolio" to open `/p/alex-rivera`. Show the theme switcher, responsive layout, and contact form.
9. **Settings & Monetization (₹59 INR)**: Open Settings, show the ₹59 INR subscription breakdown, and click "Switch to Pro (Test Mode)" to demonstrate instant server-enforced entitlement toggling.

6. Top 25 Technical Viva Questions & Examiner Answers

Q1 What is Next.js 16 App Router and why did you choose it over traditional React (Vite/CRA)?

Answer: Next.js 16 App Router provides hybrid rendering: React Server Components (RSC) for zero-client-bundle server rendering (used for SEO-critical routes like public portfolios at `/p/[slug]`) combined with Client Components for rich interactive editors. Unlike Vite or CRA which send blank HTML files requiring heavy client hydration, Next.js provides instant first contentful paint (FCP) and native API route handlers in a unified full-stack architecture.

Q2 How is user authentication implemented and secured against common vulnerabilities?

Answer: We implemented stateless JWT authentication using the `jose` library and `bcryptjs`. Passwords are salted and hashed with 12 rounds before database insertion. The signed JWT session token is stored in an `httpOnly` cookie with `SameSite: "lax"` and `secure` flags. This prevents Cross-Site Scripting (XSS) attacks from accessing the token via `document.cookie`, while the SameSite attribute mitigates Cross-Site Request Forgery (CSRF).

Q3 How does the resume live preview synchronize with the editor without performance degradation?

Answer: We separate local component state from database persistence. Keystrokes immediately update React component state in memory, triggering instant sub-tree re-rendering of the preview canvas. To prevent overwhelming the database with network requests on every keypress, we utilize a 1,200ms debounce timer before making an asynchronous `PUT` request to update the database record.

Q4 How does the AI Bullet Improver apply Google's XYZ formula?

Answer: Google's XYZ formula states: *"Accomplished [X], measured by [Y], by doing [Z]"*. Our prompt engineering pipeline enforces this structure via system instructions given to the Google Gemini model, requiring it to identify the core action (Z), extract or estimate the business impact (X), and quantify the measurable outcome (Y with percentages, dollar savings, or latency reductions) without fabricating candidate experience.

Q5 How does the ATS Keyword Matching algorithm work?

Answer: The algorithm performs lexical analysis: it lowercases both text sources, strips standard English stopwords (e.g., 'and', 'the', 'with'), and extracts unigram and bigram tokens. It then calculates the frequency and intersection between the job description's requirement tokens and the resume's content tokens. The final score is normalized between 0-100%, and unreferenced keywords are flagged as "Missing Critical Keywords" for candidate optimization.

Q6 What happens if the Google Gemini API fails, times out, or exhausts its quota?

Answer: CareerForge implements an intelligent dual-engine architecture in `src/lib/ai.ts`. The system first attempts to communicate with the Gemini API. If the call rejects (due to network timeout, HTTP 429 rate limit, or invalid credentials), the error is caught gracefully and redirected to our built-in heuristic rule-based engine, which applies local grammatical transformations and action-verb substitutions. The user experience remains seamless without errors.

Q7 How did you implement Pinterest-trending templates like Swiss Split Sidebar and Quiet Luxury?

Answer: In `src/components/ResumePreview.tsx`, we used modern CSS Grid and Flexbox layouts. For the Swiss Sidebar, we engineered a 33%/67% split where the left column features a calculated monogram initials badge and stacked SVG icon rows, while the right column features a continuous vertical SVG timeline rail with node circles positioned at each experience entry. For Quiet Luxury, we applied high-fashion serif styling with wide letter-spacing (`tracking-[0.25em]`) and numbered section indices (`01 /` , `02 /`).

Q8 How does PDF export differ from typical client-side canvas rendering (html2canvas)?

Answer: Libraries like `html2canvas` convert DOM nodes into rasterized PNG images before embedding them into a PDF. This results in blurry text on zoom, unclickable hyperlinks, huge file sizes, and broken page splits. CareerForge uses pure vector Print CSS with `@media print`, isolating the exact resume node, setting vector millimeter dimensions (A4/Letter), and invoking browser print engines to produce crystal-clear, selectable, searchable vector PDFs.

Q9 How does native Word (.docx) export work?

Answer: In `src/lib/docx-export.ts`, we use the `docx` library to programmatically generate Microsoft OpenXML document specifications. We instantiate structured Document paragraphs, heading runs, bullet point formatting, and font definitions, which are packed into a binary blob and sent with the appropriate MIME type `application/vnd.openxmlformats-officedocument.wordprocessingml.document`.

Q10 How is the personal portfolio website rendered at `/p/[slug]` ?

Answer: The route `/p/[slug]` is a dynamic Next.js App Router page. The server extracts the slug parameter, queries the database via Prisma (`prisma.portfolio.findUnique({ where: { slug } })`), increments the view counter asynchronously, and renders the selected theme (Slate, Dark Obsidian, Editorial, Creative) as responsive HTML with full OpenGraph metadata for social sharing.

Q11 What database ORM did you use and why Prisma?

Answer: We used Prisma 6.4 with SQLite. Prisma provides a declarative `schema.prisma` file that defines entities and relations (e.g., One-to-Many between User and Resumes, and One-to-One between Resume and Portfolio). Prisma auto-generates a fully typed TypeScript client, eliminating runtime SQL injection vulnerabilities through parameterized queries and catching query bugs at compile time.

Q12 How do you prevent unauthorized users from editing other people's resumes?

Answer: Every resume route handler (`GET` , `PUT` , `DELETE` in `src/app/api/resumes/[id]/route.ts`) first extracts the authenticated user ID from the verified JWT cookie. It then queries the database for the requested resume with both `id: resumeId` and `userId: authenticatedUserId`. If no matching record is found, the server immediately returns an HTTP 404/403 Forbidden response.

Q13 Explain the monetization and plan entitlement structure.

Answer: We implement a freemium model. The Free tier permits up to 3 active resumes and standard templates. The Pro tier (priced at ₹59 INR / month) unlocks unlimited resumes, all 8 premium templates (including Pinterest Swiss and Editorial Luxury), unlimited AI tailoring, and portfolio generation. All entitlements are verified server-side in API route handlers before persisting changes.

Q14 Why did you use Indian Rupees (INR ₹59) in Stripe and how is it formatted?

Answer: Stripe requires the smallest currency unit for all amounts. In Indian Rupees, 1 INR equals 100 paise. Therefore, the ₹59 subscription is represented as `unit_amount: 5900` with `currency: "inr"` in Stripe Checkout session creation parameters.

Q15 What is Tailwind CSS v4 and how does it benefit CareerForge?

Answer: Tailwind CSS v4 is a high-performance CSS-first framework built on Lightning CSS. It eliminates the traditional `tailwind.config.js` file in favor of modern CSS theme variables, compiles up to 10x faster, and enables clean utility classes for responsive breakpoints, dark mode, and custom spacing.

Q16 How does the document upload parser extract resume fields from uploaded PDFs?

Answer: The parser receives a multipart FormData file stream, reads the buffer into memory, and feeds it to `pdf-parse`. The extracted plain text is analyzed line-by-line using regex heuristics to match email patterns (`[\w.-]+@[\w.-]+\.\w+`), phone formats, standard section headers ('Experience', 'Education', 'Skills'), and date ranges to populate structured JSON.

Q17 What is a debounce function and why is it essential in autosave?

Answer: Debouncing delays the execution of a function until a specified period of inactivity has elapsed. Without debouncing, typing a 500-word paragraph would trigger 2,500 simultaneous database write requests. Our 1,200ms debounce ensures that only one consolidated network request is dispatched once the candidate pauses typing.

Q18 How do you handle password reset securely?

Answer: The user requests a reset link at `/auth/forgot-password`. The server generates a cryptographically secure random token with an expiration timestamp and saves it to the user record. A reset link containing the token is delivered. Upon submission at `/auth/reset-password`, the token is verified, the new password is hashed with bcrypt, and the reset token is invalidated.

Q19 How do you manage version history and document recovery?

Answer: In our Prisma schema, the `ResumeVersion` model stores a snapshot of the resume's complete JSON state, an optional change summary, and a timestamp. When a candidate requests a restore via `/api/resumes/[id]/versions/[versionId]/restore`, the resume data is replaced with the snapshot payload in an atomic transaction.

Q20 What are the advantages of Server-Side Rendering (SSR) for the public portfolio route?

Answer: SSR ensures that search engines (Google, Bing) and social media crawlers (LinkedIn, Twitter) receive fully populated HTML with meta tags, OpenGraph previews, and resume content without needing to execute client JavaScript. This maximizes search engine ranking and social sharing click-through rates.

Q21 How does CareerForge protect against SQL injection?

Answer: CareerForge relies exclusively on Prisma ORM for all database operations. Prisma uses parameterized queries under the hood, ensuring user inputs are treated strictly as data parameters rather than executable SQL commands, completely neutralizing SQL injection risks.

Q22 What is the difference between client-side and server-side validation in CareerForge?

Answer: Client-side validation (HTML5 forms, React state checks) provides immediate visual feedback to the user (e.g., highlighting invalid email formats). Server-side validation (in Next.js Route Handlers) is the authoritative security layer that re-validates types, lengths, and entitlements to prevent bypass via curl or automated API scripts.

Q23 How is state managed in the Next.js frontend?

Answer: Local form state is managed via React hooks (`useState` , `useCallback` , `useRef`). Server data fetching is orchestrated via asynchronous `fetch()` inside `useEffect()` . Reusable UI state (such as active modals and template selections) is lifted to parent component containers to ensure clean unidirectional data flow.

Q24 How does the test mode switcher assist during evaluation?

Answer: In **Settings** → **Subscription**, we provided an evaluator test mode switcher (`/api/billing/test-toggle`). This allows academic evaluators and examiners to toggle an account between the Free and Pro tiers instantly without entering real credit card details, enabling live demonstration of both free limits and locked Pinterest templates.

Q25 How is CareerForge deployed for production?

Answer: The codebase is pushed to GitHub at `ANNIEEONE/careerforge` . It is configured for deployment on Netlify using `@netlify/plugin-nextjs` and `netlify.toml` , running automated builds with `prisma generate && prisma db push && next build` to provide continuous integration and globally distributed CDN delivery.
